

Taller de Tecnologías de Producción de Software

Cursada 2026

Docentes

Laura Fava, Jorge Rosso, Tomás Dozo, Lourdes Valli,
Sebastián Villena, Vicente García Marti

CLASE 1 - Contenido

- Aplicaciones web. Definiciones. Arquitectura.
- Tecnologías Java
- Arquitectura de una aplicación
 - Capas lógicas (layers)
 - Capas físicas (tiers)
- Breve síntesis de HTTP y HTML
- Servidores J2EE
- Introducción a Servlets
- Parámetros de inicialización de Servlets
- Configuración mediante XML y mediante anotaciones

Aplicaciones web

Qué son y cómo trabajan?

Son programas interactivos contruidos con tecnologías web -HTML, CSS, JS- que manipulan datos que se encuentran almacenados en algún servidor. Las aplicaciones web pueden ser usadas simultáneamente por varios usuarios a través de internet.

Cualquier aplicación web típica, está compuesta por dos códigos/partes diferentes que se ejecutan en distintos lugares:

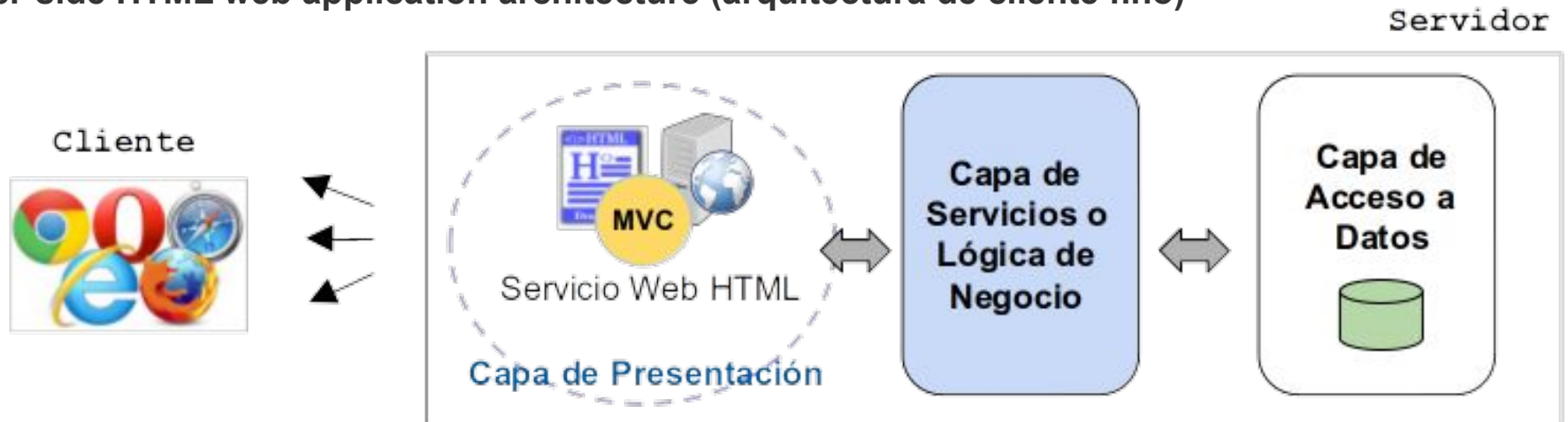
- **Frontend (client-side code):** Es el código que ejecuta en el navegador de la máquina del cliente y permite hacer peticiones y recibir/ver respuestas a esas peticiones.
- **Backend (server-side code):** Es el código que está en el o los servidores, los cuales procesan y responden a los requerimientos desde los clientes.



Aplicaciones web

Tipos de arquitecturas

Server-side HTML web application architecture (arquitectura de cliente fino)



Service-oriented single-page web apps (arquitectura de cliente grueso)



Arquitectura de una aplicación

Layers (capas lógicas) y Tiers (capas físicas)

Layers

Son formas de organizar el código. Una aplicación, típicamente incluye como mínimo las capas de Presentación, Negocio y Datos.

¿Por qué usar layers?

- Para lograr beneficios de una organización lógica y agrupar por funcionalidad => reusabilidad.
- Reduce los costos de mantenimiento de la aplicación.
- Mejor diseño.

Tiers

Se refiere a donde corre el código. Las *tiers* son los lugares donde ejecutan las *layers*. Cada recurso ejecutando en un proceso es considerado un *tier*.

¿Cuál es el beneficio de hacer el despliegue (deploy) y la ejecución de las capas lógicas en las capas físicas?

Para lograr escalabilidad, tolerancia a fallas y seguridad.

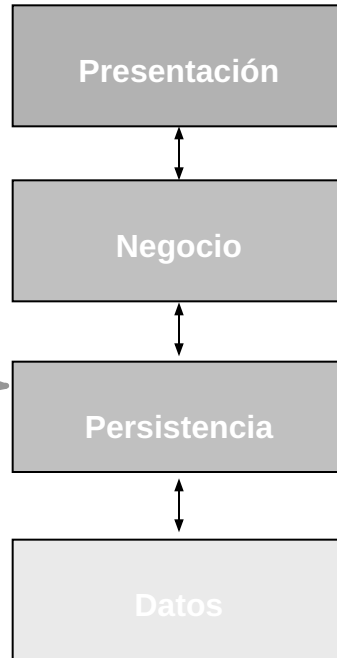
Arquitectura de una aplicación

Layers (capas lógicas)

La mayoría de las aplicaciones pueden ser organizadas para que soporten un conjunto de capas lógicas o layers. Una arquitectura de alto nivel para una aplicación empresarial, usa estas capas:

Arquitectura
Layered

Cada lógica (**layer**) provee un conjunto de servicios a la/s capa/s adyacentes.



Provee la lógica necesaria para la interfaz de usuario. Está destinada a formatear y desplegar información.

Implementa la lógica necesaria de la aplicación. En esta capa se implementan las reglas de negocio o requerimientos del sistema.

Esta capa está formada por un conjunto de clases y componentes responsables de almacenar los datos y recuperarlos desde una fuente de datos.

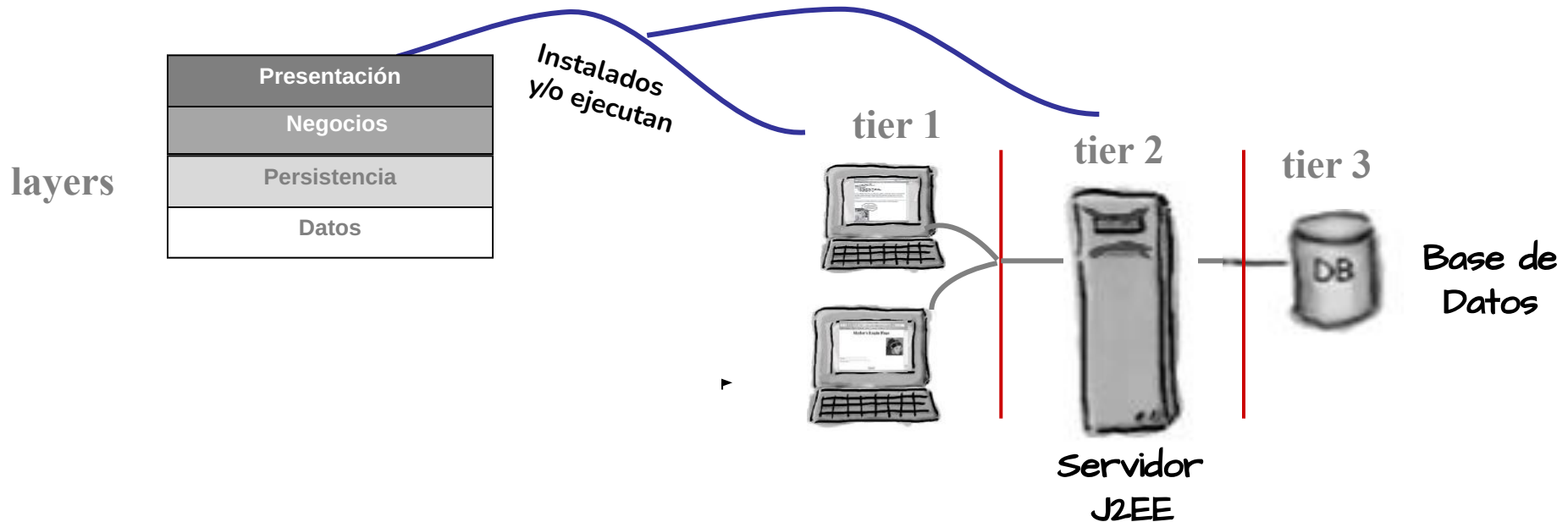
Representa los datos subyacentes. Es la representación persistente de los datos del sistema.

Con una arquitectura *layered*, los cambios en una de las capas no deberían afectar o afectar muy poco al resto de las capas.

Arquitectura de una aplicación

Tiers (capas físicas)

Una plataforma *empresarial* es por naturaleza, una plataforma distribuida. Las tareas están distribuidas en distintas computadoras. Cada computadora implementa/ejecuta uno o más **layers**.



Una aplicación JEE se “despliega” comúnmente en una máquina cliente, un servidor JEE y un servidor de datos.

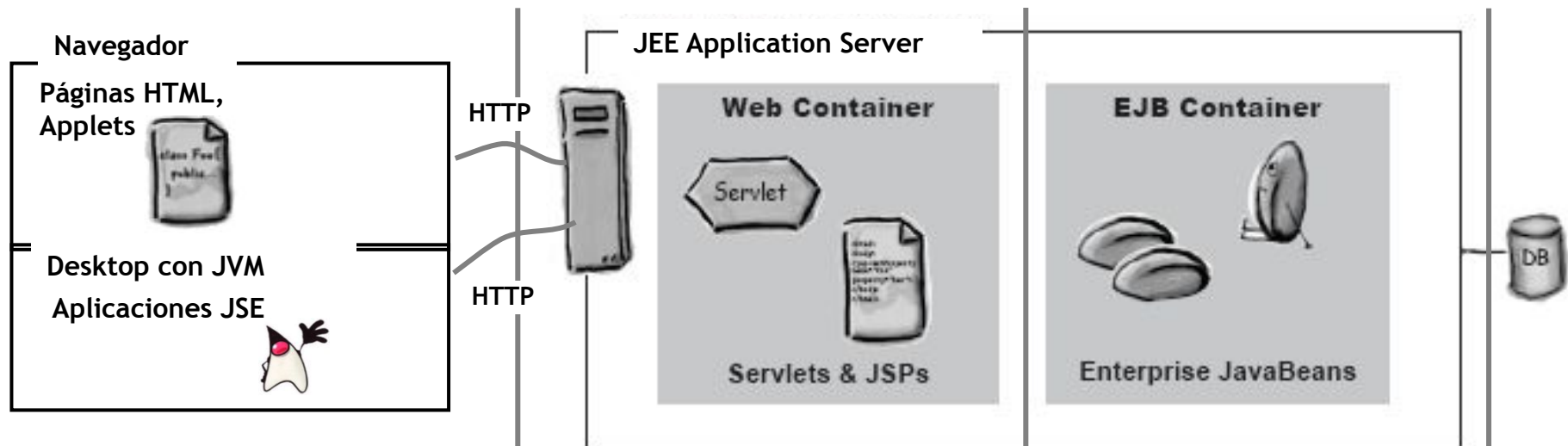
Arquitectura de JEE (Jakarta EE Platform)

Componentes

La plataforma JEE usa un modelo de aplicación “multitiered” distribuida. La lógica de la aplicación está dividida en componentes de acuerdo a su función, y las componentes que forman parte de la aplicación java EE son instaladas en varias máquinas dependiendo de la “tier” en el ambiente “multitiered” java EE a la que pertenece.

La especificación Java EE define las siguientes componentes:

- **Componentes Cliente:** Aplicaciones JSE, páginas HTML.
- **Componentes Web:** Servlets, Filtros, Listeners, JSPs y JavaServer Faces
- **Componentes Empresariales:** Enterprise JavaBeans



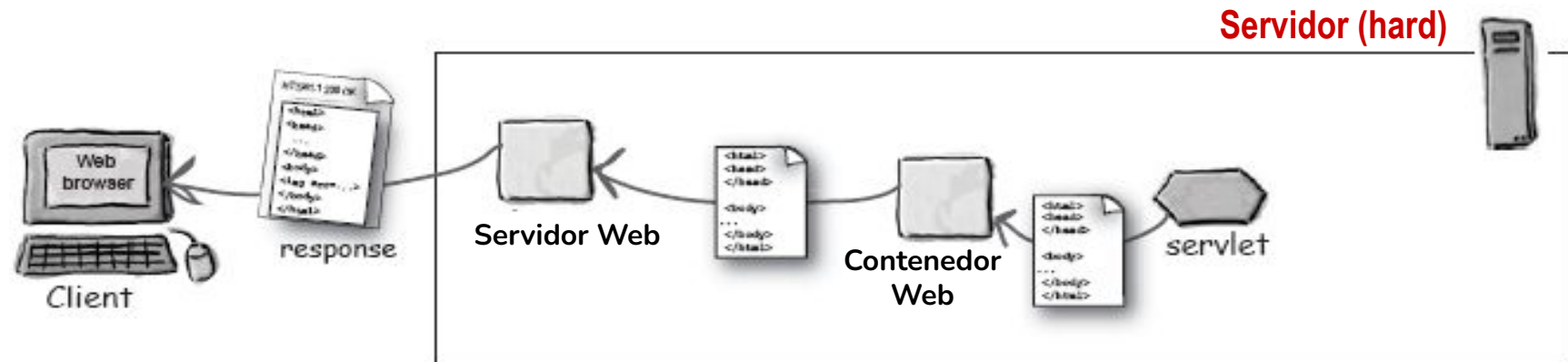
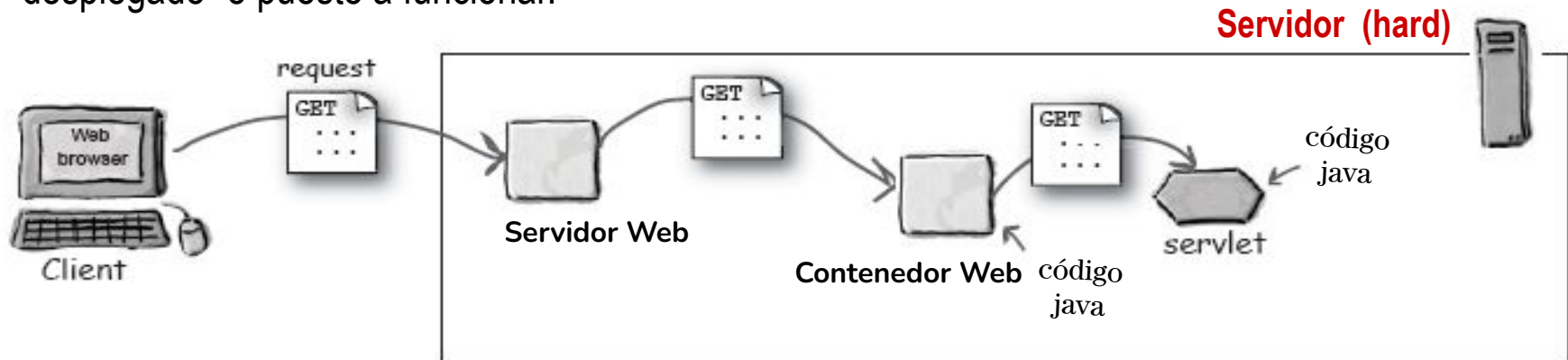
Una aplicación JEE está basada en componentes, donde una componente JEE es una unidad funcional de software, que debe correr dentro de un contenedor y que puede comunicarse con el resto de las componentes.

Arquitectura JEE

Contenedores

Las componentes web JEE no tienen un método `main()` como las aplicaciones de escritorio. Ellas están bajo el control de otra aplicación java, llamada Contenedor.

Cuando el servidor HTTP recibe un requerimiento para una componente web (supongamos un Servlet), el servidor no maneja el requerimiento solo, sino que lo hace con ayuda del contenedor web en donde el servlet fue “desplegado” o puesto a funcionar.



Arquitectura JEE

Contenedores y Servidores JEE

Servidores JEE certificados según la especificación Jakarta EE:

Contenedor JEE	Tomcat (Apache) Jetty (Eclipse Foundation)  Apache Tomcat
Servidores de aplicaciones JEE	TomEE (Apache) GlassFish (Eclipse Foundation) Red Hat JBoss EAP (Enterprise Application Platform) WebSphere Application Server (IBM)  Apache TomEE



Arquitectura JEE

Contenedores y Servidores JEE - Tomcat o TomEE

Apache Tomcat y **Apache TomEE** son dos alternativas de código abierto para ejecutar aplicaciones Java web, pero tienen alcances diferentes.

Apache Tomcat

Tomcat es principalmente un servidor web y contenedor de Servlets. Implementa un subconjunto de las especificaciones de Jakarta EE, incluyendo tecnologías como: Jakarta Servlet, Jakarta Pages (JSP), Expression Language (EL), WebSocket, Jakarta Authentication. La rama actual, Tomcat 11, implementa las especificaciones correspondientes a Jakarta EE 11, entre ellas Servlet 6.1 y Pages 4.0, y requiere Java 17 o superior.

Tomcat se caracteriza por ser relativamente liviano, tener un inicio rápido y ser ampliamente utilizado como servidor embebido en frameworks como Spring Boot.

Apache TomEE

TomcatEE parte de Apache Tomcat y agrega tecnologías de Jakarta EE para proporcionar un servidor de aplicaciones más completo. Según la distribución utilizada, incorpora tecnologías como:

CDI (Contexts and Dependency Injection), JPA (Jakarta Persistence), JTA (Jakarta Transactions), EJB (Enterprise Beans), Jakarta REST (JAX-RS), Jakarta Faces (JSF), Jakarta Messaging (JMS), Jakarta Web Services.

La versión estable TomEE 10 implementa Jakarta EE 10.

Tanto Tomcat como TomEE son proyectos de la **Apache Software Foundation (ASF)** y utilizan **Apache License 2.0**.

Cientes y servidores

conocen HTTP y HTML

¿Qué es el protocolo HTTP?

La mayoría de las conversaciones que mantienen los navegadores con los servidores lo hacen usando el protocolo HTTP -HyperText Transfer Protocol-, el cual permite conversaciones del tipo requerimiento - respuesta. El cliente envía un requerimiento HTTP y el servidor responde con una respuesta HTTP.

Los elementos clave de un **request**:

- El **método HTTP**
- El recurso a acceder (**URL**)
- Los **parámetros** del formulario



¿Qué es HTML?

Cuando un servidor responde a un requerimiento, en general envía al navegador un conjunto de instrucciones escritas en HTML -HyperText Markup Language-. El HTML le dice al navegador **cómo** mostrar el contenido al usuario.

Los elementos clave de un **response**:

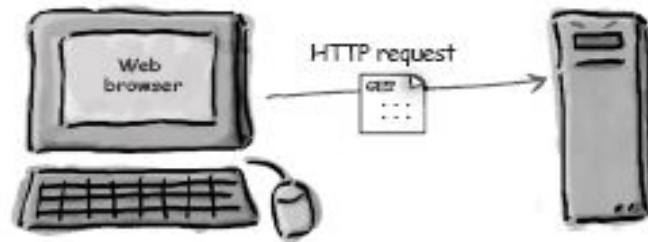
- El código de estado.
- El Content-type (text-picture-HTML, etc.)
- El contenido

En términos generales los códigos de estados 5xx son errores de servidor, los 4xx son errores de clientes, los 3xx son redirecciones y los 2xx ok

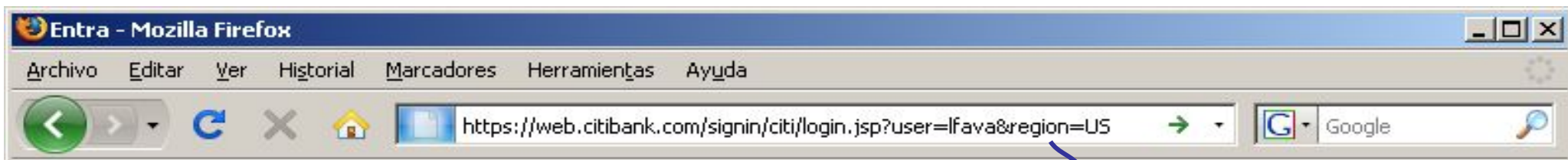
HTTP Request

¿Qué tiene un requerimiento HTTP?

Un requerimiento HTTP tiene un nombre de método (no son métodos java). HTTP tiene varios métodos pero básicamente usaremos GET/POST.



Los métodos más comunes para hacer una petición a un determinado recurso son GET/POST. Ambos pueden enviar datos en el pedido: con el GET los datos aparecen en la URL y tiene limitaciones –cada servidor soporta una cantidad limitada de caracteres-, con POST no hay límite porque los datos viajan en el cuerpo del HTTP.



Cuando se tipea la URL en la barra del navegador (al igual que al presionar un link) se genera un método GET

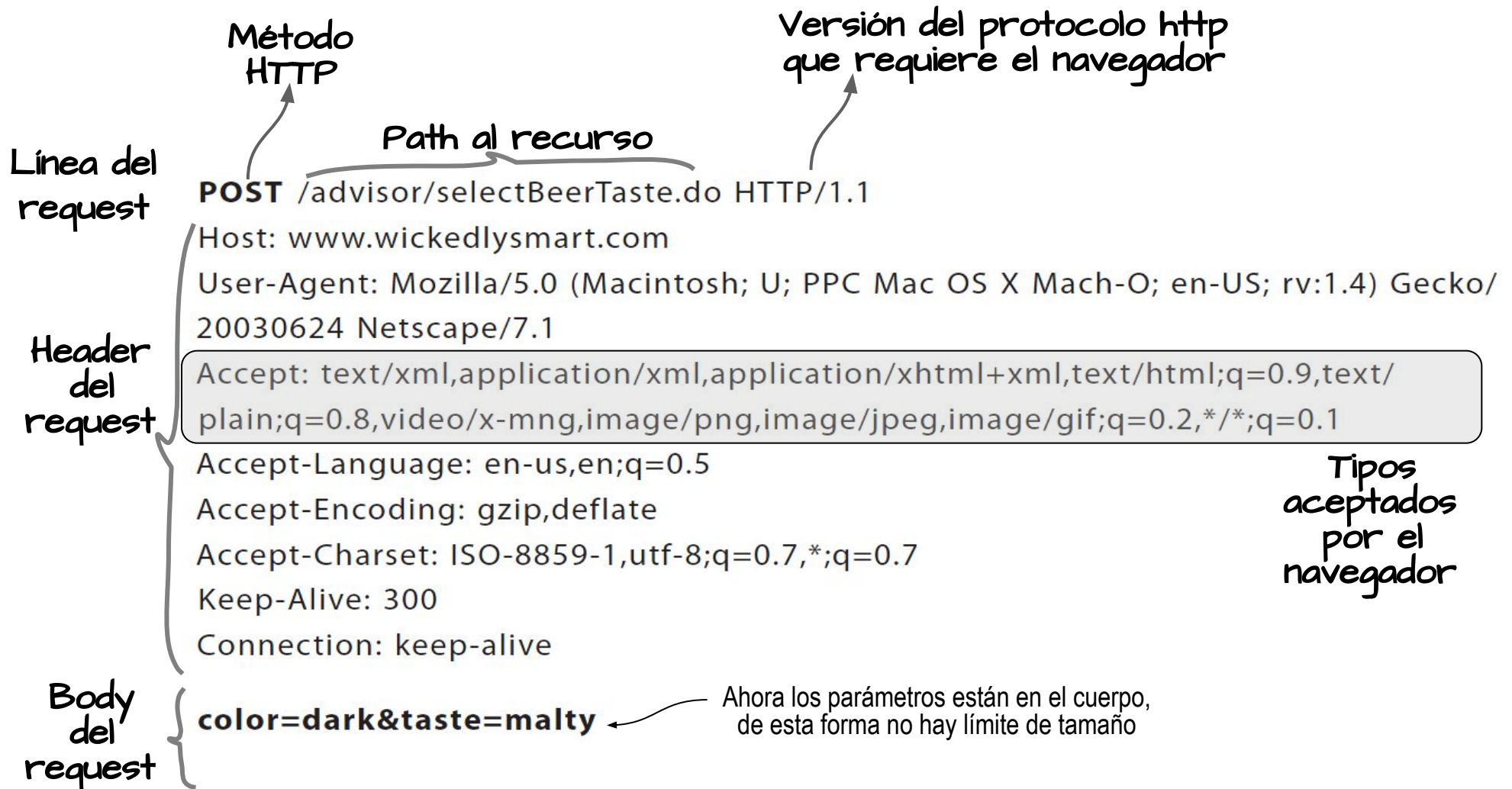
HTTP Request

Anatomía de un requerimiento HTTP GET



HTTP Request

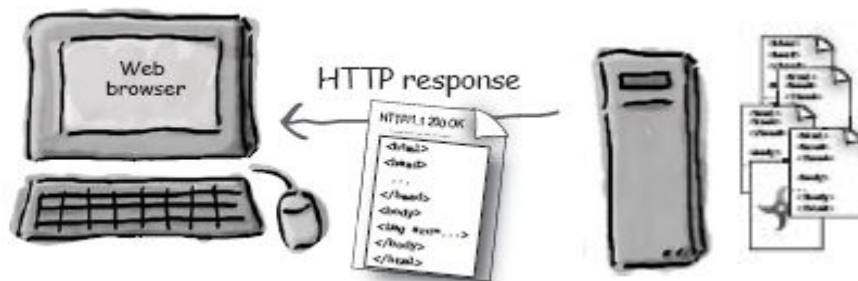
Anatomía de un requerimiento HTTP POST



HTTP Response

¿qué tiene una respuesta HTTP?

Una respuesta HTTP *puede* contener HTML. HTTP agrega un **header** con información arriba del contenido en la respuesta. Un navegador usa el **header** para procesar la respuesta y poder mostrarla.



HTTP header

HTTP header info

```
<HTML>
<HEAD>
<TITLE> Ejemplo de HTML </TITLE>
</HEAD>
<!-- cuerpo -->
<BODY>
```

Comentario HTML

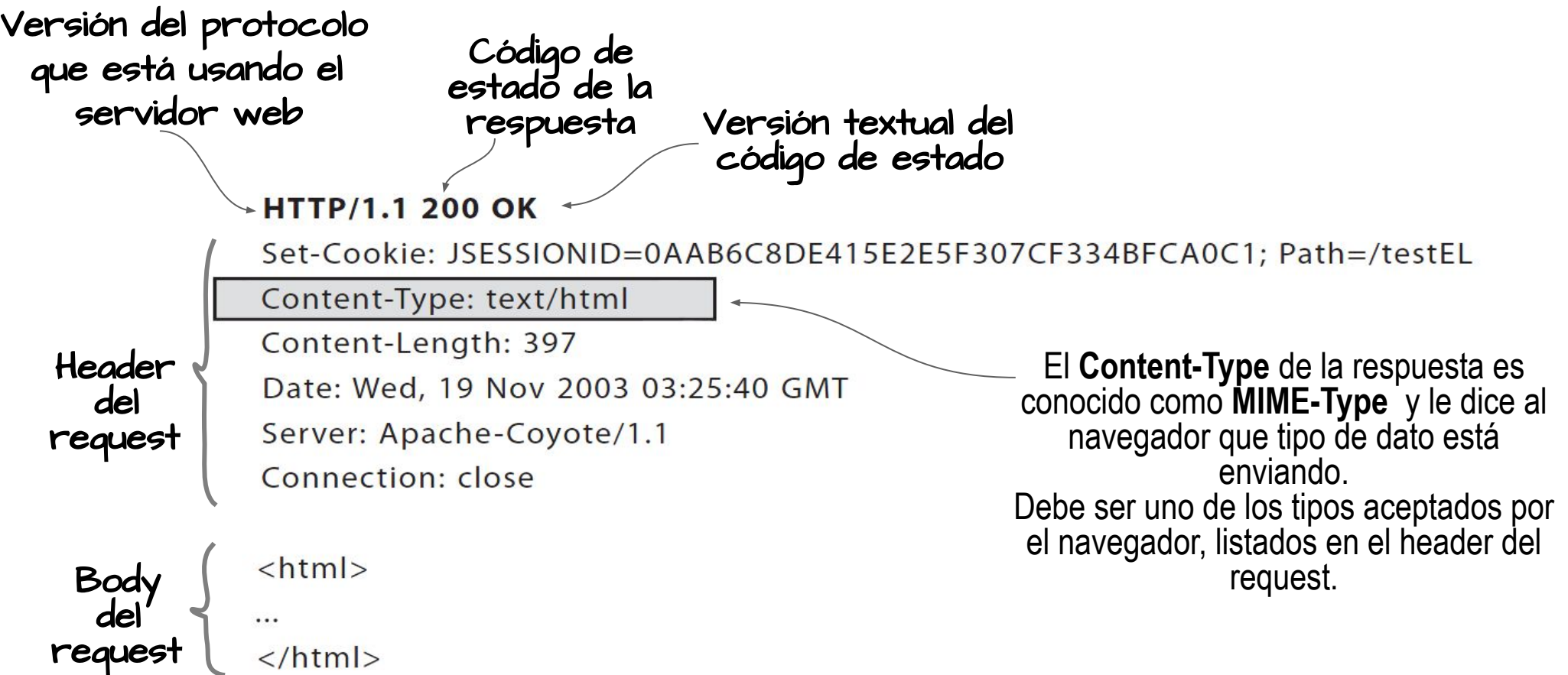
HTTP body

```
<P>
  Esto es un ejemplo de un documento HTML.
</P>
<FORM ACTION="chequearLogin" METHOD="POST">
...
</FORM>
</BODY>
</HTML>
```

Estructura de un documento HTML.
Es parte de la respuesta HTTP

HTTP Response

Anatomía de un response HTTP

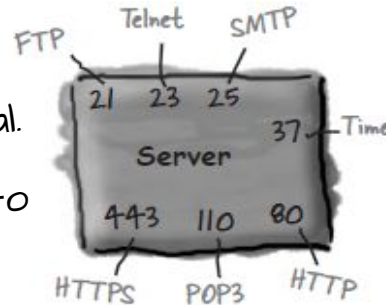


URL: Uniform Resource Locators

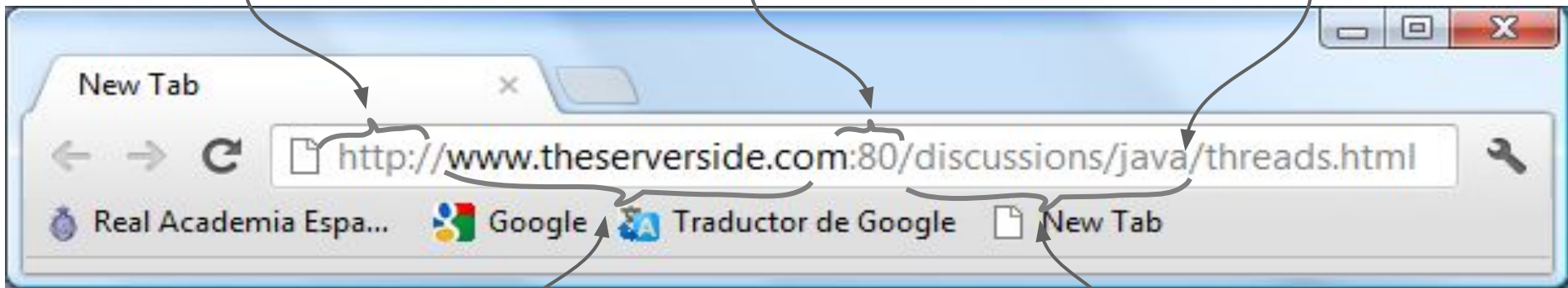
Cada recurso en la web tiene su propia y única dirección en el formato URL.

Protocolo : le indica al servidor que protocolo de comunicación será utilizado (http/https)

Port (puerto) : esta parte de la url es opcional. Cada servidor soporta muchos puertos. El puerto por defecto para servidores web, es 80.



Recurso : el nombre del recurso que se está pidiendo. Podría ser un HTML, una imagen, un servlet, etc. Si no se especifica nada, el servidor buscará un index.html



Servidor : nombre único del servidor al que se hace la petición. Este nombre mapea con una única dirección IP y se podría especificar la dirección IP en vez del nombre.

Path: camino de la ubicación del recurso buscado en el servidor.

Query String: si es un requerimiento GET la información extra (parámetros) son agregados al final de esta URL, comenzando con "?" y cada uno de los parámetros (nombre=valor) separado por "&".

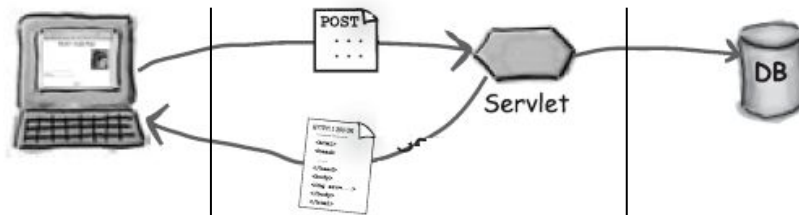
Servlets

Características generales

Servlets

Características generales

- Un servlet es una componente web escrita en Java que es gerenciada por un Contenedor Web. Procesa requerimientos y construye respuestas dinámicamente. Son ideales para realizar procesamiento en la capa del medio (middleware). Es la tecnología básica para la construcción de aplicaciones web JAVA.



- Los servlets son clases Java independientes de la plataforma, se compilan a código de bytes (bytecodes), se cargan dinámicamente y se ejecutan en un Contenedor web.
- Los servlets hacen uso de la Plataforma Java y de servicios provistos por el Contenedor Web
- La Plataforma Java provee una API robusta basada en POO que se puede usar para construir servlets.
- El Contenedor Web, evita que el programador se ocupe de la conectividad con la red, de capturar los pedidos, de producir las respuestas, seguridad, etc. Gerencia el ciclo de vida del servlet.
- La última versión de **Servlet es la 6.1**, liberada con la plataforma **JEE 11** (<https://jakarta.ee>).

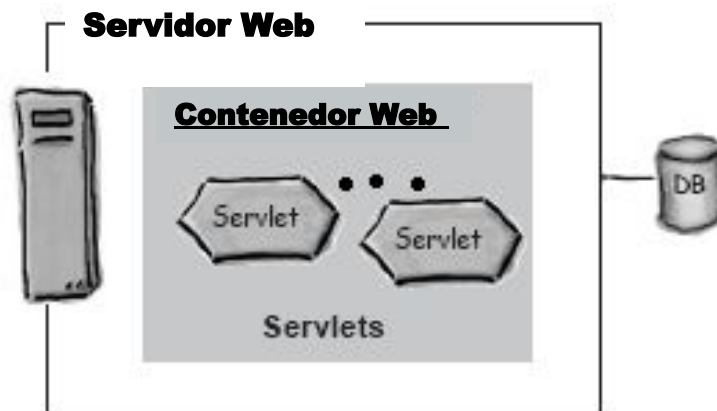
Servlets

El Contenedor Web

El Contenedor Web es responsable de:

1. la conectividad con la red
2. capturar los requerimientos HTTP, traducirlos a objetos que el servlet entienda, entregarle dichos objetos al servlet quién los usa para producir las respuestas
3. generar una respuesta HTTP en un formato correcto (MIME-type)
4. gerenciar el ciclo de vida del servlet
5. manejar errores y proveer seguridad

Un Contenedor Web es una extensión del servidor web que provee soporte para servlets. Es parte del servidor web o del servidor de aplicaciones.



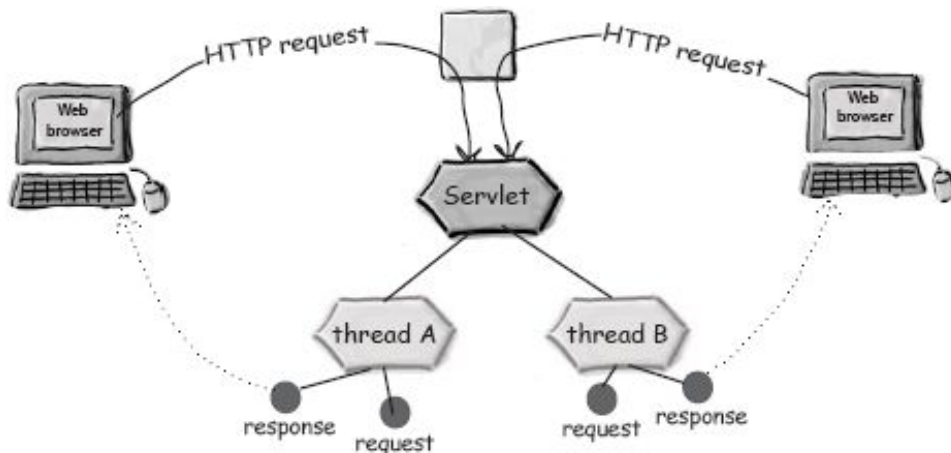
El Contenedor Web interactúa con los servlets invocando métodos de gerenciamiento o métodos callback. Estos métodos definen la interfaz entre el Contenedor y los servlets (API de servlets).

El Contenedor Web está construido sobre la plataforma estándar de Java (JSE™), implementa la API de servlets y todos los servicios requeridos para procesar pedidos HTTP. Cada contenedor web tiene su propia implementación de la API de servlets.

Contenedor Web

Los servlet son controlados por el Contenedor

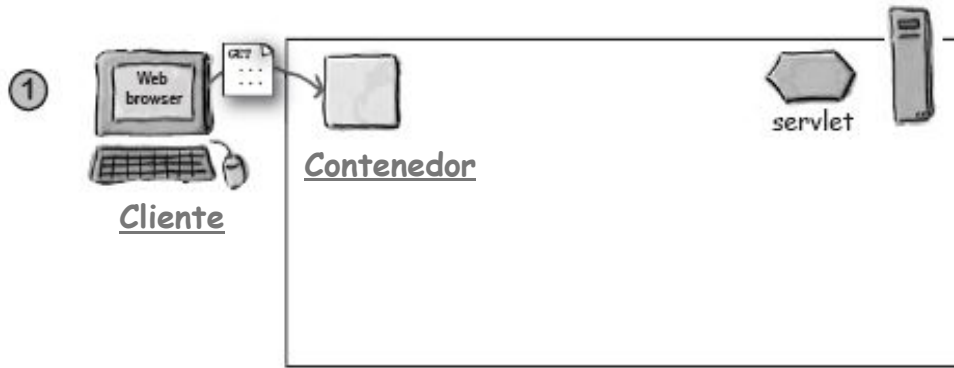
- El Contenedor Web gerencia el ciclo de vida de cada servlet, invocando a 3 métodos definidos en la interface `javax.servlet.Servlet: init(..)`, `service(..)` y `destroy(..)`
- El Contenedor Web, es responsable de la carga e instanciación de los servlets, que puede suceder en el arranque, cuando una aplicación es actualizada (recargada) o puede ser postergada hasta que el servlet es requerido por primera vez.
- El Contenedor Web, crea una única instancia de cada servlet declarado en la aplicación web.
- El Contenedor Web, maneja los requerimientos concurrentes a un mismo servlet, ejecutando el método `service()` concurrentemente en múltiples threads Java.



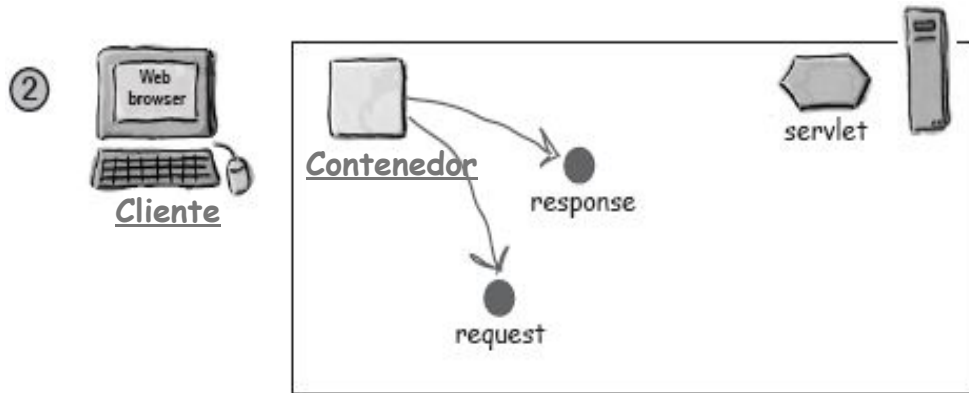
El programador de servlets, debe considerar los efectos colaterales que tiene el procesamiento concurrente y tomar las previsiones necesarias sobre los recursos compartidos (acceso a variables de instancia, variables de clase, recursos externos como archivos, etc.)

Contenedor Web

Los servlet son controlados por el Contenedor

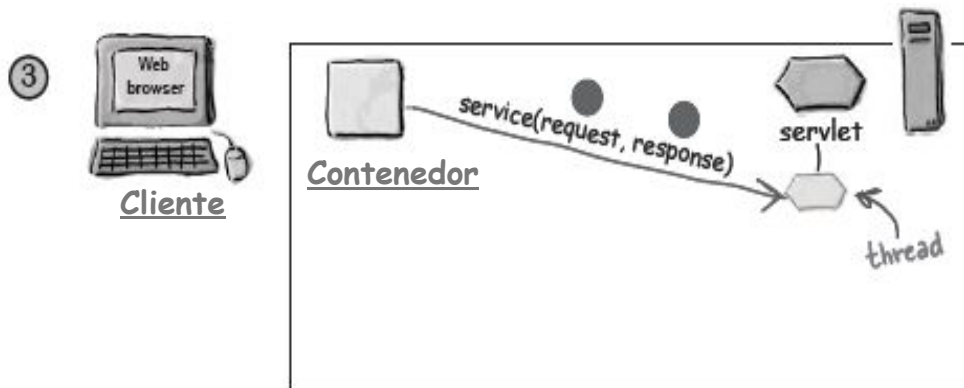


El usuario presiona sobre un link que tiene una URL a un Servlet o se invoca a través de un formulario



El Contenedor crea 2 objetos:

- 1) `HttpServletResponse`: representa la respuesta que se le enviará al cliente
- 2) `HttpServletRequest`: representa el requerimiento del cliente.



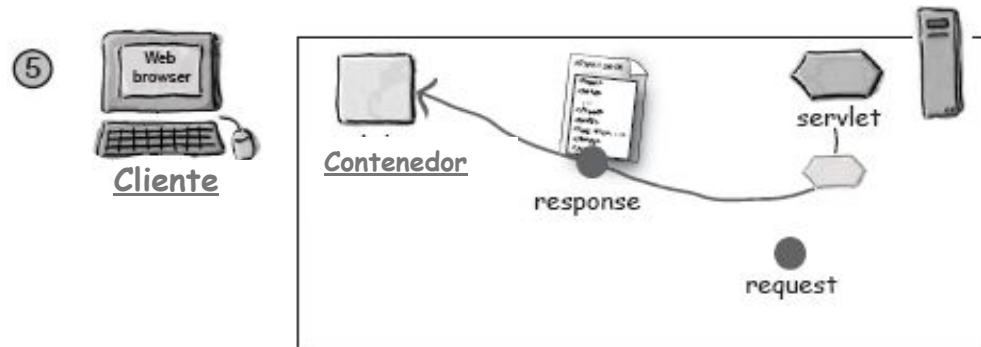
El Contenedor encuentra el servlet correcto usando la URL del requerimiento. Luego crea o aloca un thread para ese requerimiento e invoca al método `service()` pasándole los objetos `request` y `response` como argumento.

Contenedor Web

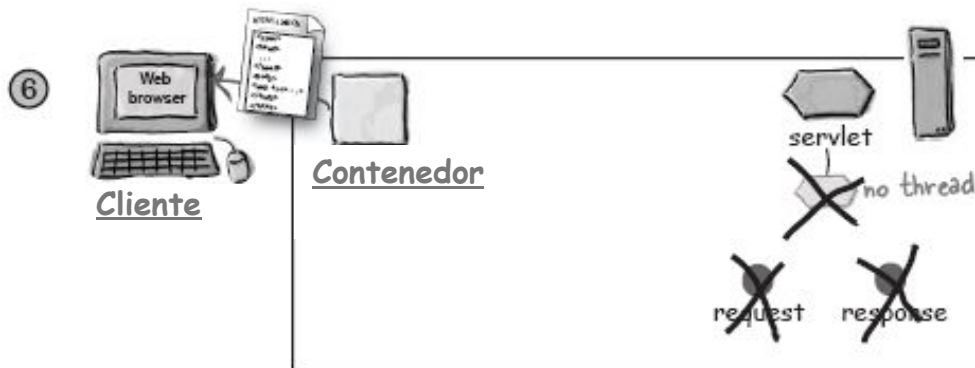
Los servlet son controlados por el Contenedor



El contenedor invoca al método `service()` del servlet, el cual determina que método invocar dependiendo del método HTTP (get o post) enviado por el cliente.



El servlet usa el objeto `response` para escribir la respuesta al cliente. La respuesta regresa a través del contenedor.



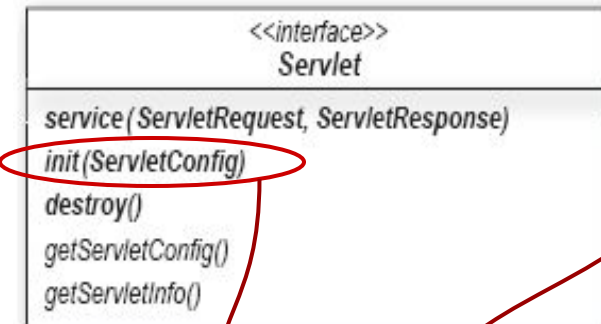
El método `service()` finaliza. El thread termina o retorna al pool de threads del contenedor. El cliente recibe la respuesta.

Servlets

Los servlets heredan los métodos del Ciclo de Vida

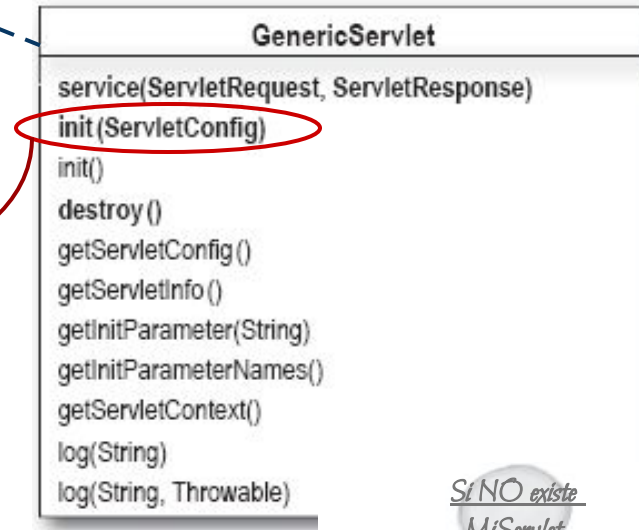
La interface Servlet
(`javax.servlet.Servlet`)

Todos los servlet tienen estos métodos, los 3 primeros son los del ciclo de vida.



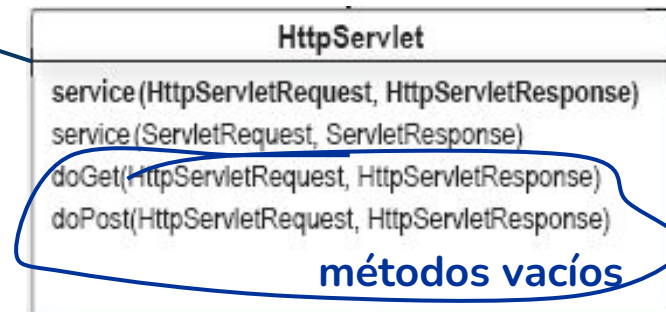
El Contenedor crea un objeto `ServletConfig` que es pasado como parámetro al método `init` y de esta manera el servlet puede acceder a parámetros de inicialización (de la forma nombre-valor)

La clase `GenericServlet`
(`javax.servlet.GenericServlet`)
Es una clase abstracta que implementa los métodos de la interface `Servlet`.

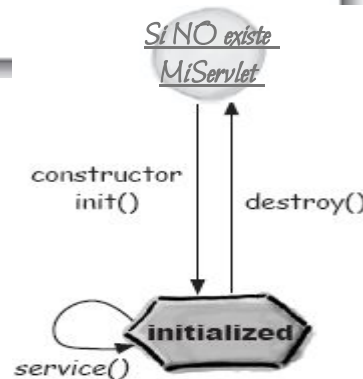


La clase `HttpServlet`
(`javax.servlet.http.HttpServlet`)

Es una clase abstracta que implementa el método `service()` para que decida qué método invocar, basado en el método HTTP.



Se sobrescriben los métodos necesarios.



Interfaces de programación

La interfaz Servlet – La clase HttpServlet

Las clases e interfaces para implementar servlets están agrupadas en dos paquetes:

- **javax.servlet**: contiene la interfaz básica de servlets, llamada **Servlet**, la cual es la abstracción central de la API de servlets.

```
public void init (ServletConfig config) throws ServletException
public void service(ServletRequest req, ServletResponse res) throws . . .
public void destroy()
public ServletConfig getServletConfig()
public String getServletInfo()
```

- **javax.servlet.http**: contiene la clase **HttpServlet** que implementa la interface Servlet y una serie de clases e interfaces específicas para atender requerimientos HTTP. La clase HttpServlet provee una implementación específica para HTTP de la interface javax.servlet.Servlet. Agrega métodos adicionales, que son invocados automáticamente por el método **service()** para ayudar al procesamiento de requerimientos HTTP. Es la clase a partir de la cual se crean la mayoría de los servlets HTTP.

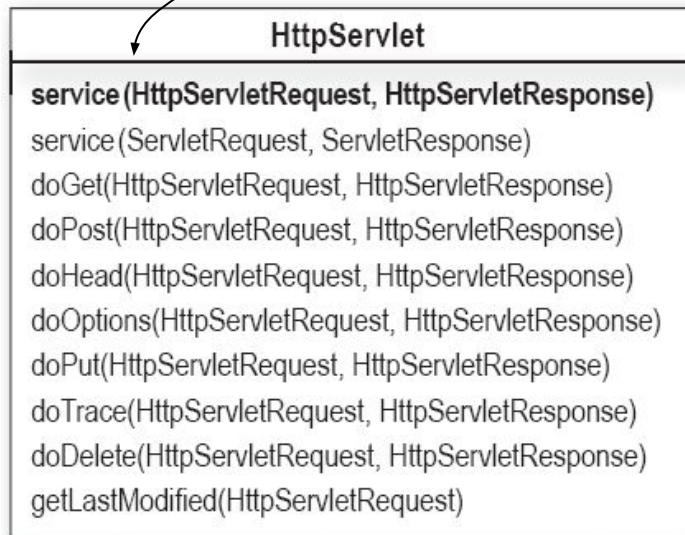
```
public void service(HttpServletRequest req, HttpServletResponse res) throws . . .
```

Interfaces de programación

La clase HttpServlet – El método service(...)

¿Cómo funciona el método **service(...)** de la clase HttpServlet?

El método **service(...)** mapea cada método del requerimiento HTTP con un método java de la clase HttpServlet.



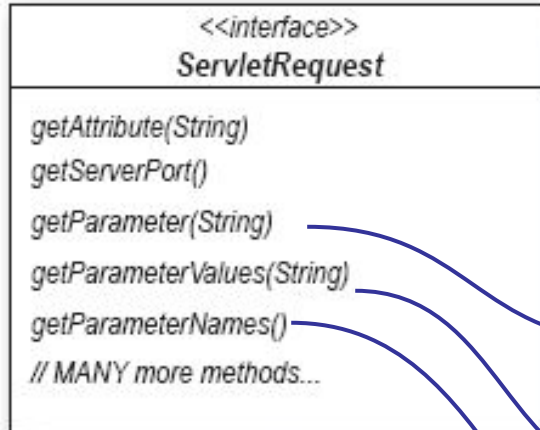
- La secuencia de métodos **service()**-->**doGet()** ó **service()**-->**doPost()** sucede cada vez que un cliente hace un requerimiento.
- Cada vez que un **doGet()** o **doPost()** ejecutan, lo hacen en un thread separado.
- Cuando se escribe un Servlet, los métodos que se sobrescriben son **doGet()** y **doPost()**, los restantes están relacionados con la programación más cercana al protocolo HTTP.

Interfaces de programación

Las interfaces `HttpServletRequest`

`ServletRequest` interface

(`javax.servlet.ServletRequest`)



El requerimiento HTTP de un cliente está representado por un objeto **`HttpServletRequest`**. Un objeto **`HttpServletRequest`** se puede usar para recuperar el header del requerimiento HTTP, recuperar los parámetros del requerimiento HTTP, asociar atributos con el requerimiento, redireccionar requerimientos entre servlets, recuperar la sesión del usuario, etc.

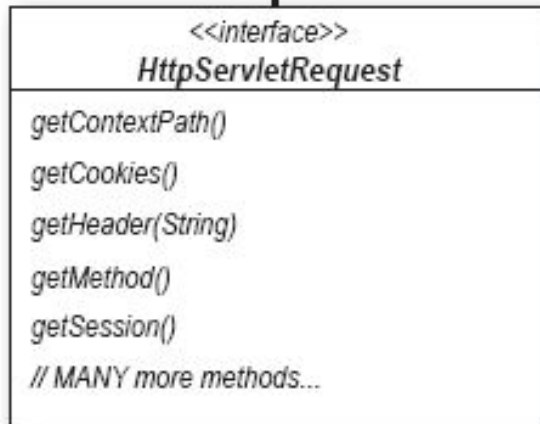
Devuelve un **`String`** con el valor del parámetro del requerimiento con la clave dada. Si hay múltiples valores para esa parámetro, devuelve el primero.

Retorna un arreglo de **`Strings`** que contiene todos los valores de un parámetro del request con la clave dada o null si el parámetro no existe.

Devuelve un List de **`Strings`** con los nombres de todos los parámetros del requerimiento.

`HttpServletRequest` interface

(`javax.servlet.http.HttpServletRequest`)

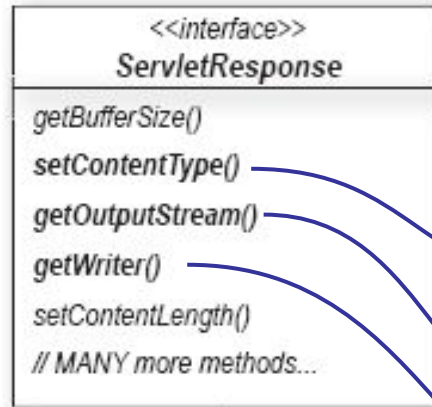


Interfaces de programación

Las interfaces HttpServletResponse

ServletResponse interface

(javax.servlet.ServletResponse)



El objeto **HttpServletResponse** representa la respuesta que se le enviará al cliente. Por defecto, la respuesta HTTP está vacía. Para generar una respuesta customizada, es necesario usar los métodos **getWriter()** o **getOutputStream()**, para obtener un stream de salida donde escribir contenido.

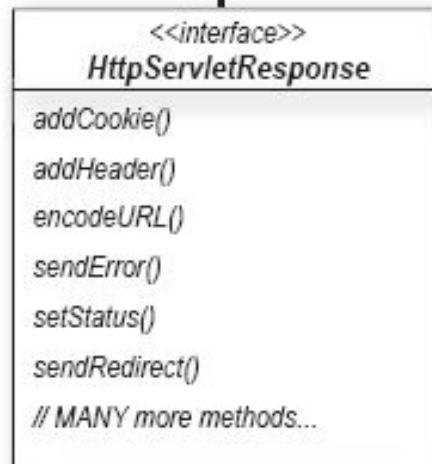
Permite setear el tipo MIME de la respuesta HTTP (text/html, image/JPG, etc.) antes de devolver la respuesta.

El objeto **ServletOutputStream** es usado para enviar al cliente datos binarios (imágenes por ejemplo).

El objeto **PrintWriter** que devuelve, es usado por el servlet para escribir la respuesta como texto.

HttpServletResponse interface

(javax.servlet.http.HttpServletResponse)



Un ejemplo simple

Un servlet que recupera parámetros del requerimiento (con XML)

Este Servlet genera una página HTML usando un parámetro del requerimiento

saludo.html

```
<html>
<body>
<form action="ServletHola" method="post">
  Ingresá tu nombre: <input type="text" name="nombre">
  <input type="submit" value="Enviar">
</form>
</body>
</html>
```

web.xml

```
. . .
<servlet>
  <servlet-name>MiServlet</servlet-name>
  <servlet-class>servlets.ServletHola</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>MiServlet</servlet-name>
  <url-pattern>/ServletHola</url-pattern>
</servlet-mapping>
. . .
```

```
package servlets;

public class ServletHola extends javax.servlet.http.HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h1> Hola " + request.getParameter("nombre")+" </h1>");
        out.print(" </body></html>");
        out.close();
    }
}
```

A partir del objeto response se puede obtener un objeto PrintWriter que nos permite escribir texto HTML en la respuesta

A partir del objeto request se pueden obtener los parámetros del requerimiento

Anotaciones en JAVA

A partir de la versión de Servlet 3.0 se pueden utilizar anotaciones para la configuración de los Servlets. Las anotaciones en la API de servlets se utilizan para reemplazar a las declaraciones/los mapeos del archivo `web.xml`.

¿Qué son las anotaciones?

- Las anotaciones son metadatos que nos permiten agregar información a nuestro código fuente para ser usado posteriormente –en tiempo de compilación o en tiempo de ejecución-.
- Las anotaciones fueron incorporadas al lenguaje java en la versión 5. La motivación de las anotaciones es la tendencia a combinar metadatos con código fuente, en lugar de mantenerlos en archivos descriptores separados.

La API de Servlets 3.0 o superior

Anotaciones

Las anotaciones **se declaran** de manera parecida a las interfaces, solo que el signo **@** precede a la palabra clave **interface**. Se compilan a archivos **.class** de la misma manera que las clases e interfaces.

```
package java.lang;
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Override {
}
```

Las anotaciones **se utilizan** en el código fuente precediendo a la declaración de la clase, atributos y métodos. En este caso, la anotación **@Override** es para métodos y se usa así:

```
public class Paciente {
    @Override
    public String toString() {
        return super.toString();
    }
}
```

Precede a un
método



La anotación **@Override** indica que se está sobrescribiendo un método de la superclase. Si un método está precedido por esta anotación pero NO sobrescribe el método de la superclase, los compiladores deben generar un mensaje de error y la clase no compila.

@Target: indica dónde se aplican las anotaciones (métodos, clases, variables de instancia, variables locales, paquetes, constructores, etc.)

@Retention: indica dónde están disponibles las anotaciones y cuánto se mantiene la información de las anotaciones. Esto permite determinar si pueden ser leídas sólo por el compilador o por el intérprete en tiempo de ejecución. Los valores posibles son: **RetentionPolicy.SOURCE**, **RetentionPolicy.CLASS** y **RetentionPolicy.RUNTIME**.

La API de Servlets 3.0 o superior

Anotaciones

Este es un ejemplo de una declaración de la anotación `@Column` para el mapeo de objetos con tablas de una base de datos, donde tiene entre otros el método `name()` para identificar en nombre de la columna en la tabla de la base de datos.

Definición de la anotación `@column`

```
import javax.persistence.*;
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@interface Column extends Annotation{

    public String name() default "";
    . . .
}
```

La declaración de una anotación al igual que las interfaces tiene métodos abstractos pero además puede tener valores por defecto.

Uso de la anotación `@column`

```
package taller;

import javax.persistence.*;

@Entity
@Table(name="MENSAJES")
public class Mensaje {

    @Column(name="MENSAJE_ID")
    private Long id;
    . . .
}
```

Preceden a la declaración de la clase

La anotación tiene una lista entre paréntesis de pares **elemento-valor**. Los valores de los elementos deben ser constantes definidas en compilación

Un ejemplo simple

Un servlet que recupera parámetros del requerimiento (con anotaciones, Servlet 3.0+)

```
<html>                                     saludo.html
<body>
<form action="ServletHola" method="post">
  Ingresá tu nombre: <input type="text"
name="nombre">
  <input type="submit" value="Enviar">
</form>
</body>
</html>
```

- La anotación `@WebServlet` es usada para declarar la configuración de un Servlet. Si no se especifica el atributo `name` se usa el nombre de la clase
- El atributo `urlPatterns` define uno o un conjunto de urlpatterns que pueden ser usadas para invocar al Servlet.

```
package misServlet;
@WebServlet(
    urlPatterns = {"/ServletHola"},
}
public class ServletFecha extends HttpServlet{

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
                                                                    ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1> Hola " + request.getParameter("nombre")+" </h1>");
        out.print(" </body></html>");
        out.close();
    }
}
```

Servlets

Parámetros de Inicialización

Parámetros de Inicialización

ServletHola con XML

Este Servlet retorna una página HTML con un mensaje concatenando un parámetro de inicialización con un parámetro del requerimiento.

```
public class ServletHola extends HttpServlet {
    private String saludo;
    public void init(){
        saludo = this.getServletConfig().getInitParameter("saludo");
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ..{
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>"+saludo+request.getParameter("nombre")+ " </h1>");
        out.println("</body></html>");
        out.close();
    }
}
```

También se podría hacer sin sobrescribir el método `init()` y recuperar los valores de inicialización cuando se quiera usar, de alguna de estas dos maneras:

```
this.getInitParameter("saludo")
```

```
. . .
<servlet>
  <servlet-name>MiServlet</servlet-name>
  <servlet-class>servlets.ServletHola</servlet-class>
  <init-param>
    <param-name>saludo</param-name>
    <param-value>Hola</param-value>
  </init-param>
</servlet>
<servlet-mapping>
  <servlet-name>MiServlet</servlet-name>
  <url-pattern>/ServletHola</url-pattern>
</servlet-mapping>
. . .
```

web.xml

Parámetros de Inicialización

ServletHola con anotaciones

Este Servlet retorna una página HTML con un mensaje concatenando un parámetro de inicialización con un parámetro del requerimiento.

```
package misServlet;

@WebServlet(
    urlPatterns = {"/ServletHola"},
    initParams = {
        @WebInitParam(name = "saludo", value = "Hola")
    })
public class ServletHola extends HttpServlet{
    private String saludo;
    public void init(){
        saludo = this.getServletConfig().getInitParameter("saludo");
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ..{
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>"+saludo+request.getParameter("nombre")+" </h1>");
        out.println("</body></html>");
        out.close();
    }
}
```

La anotación `@WebInitParam` se usa para definir los parámetros de inicialización del servlet

Atributos y Alcances



¿Qué es un atributo y dónde pueden guardarse?

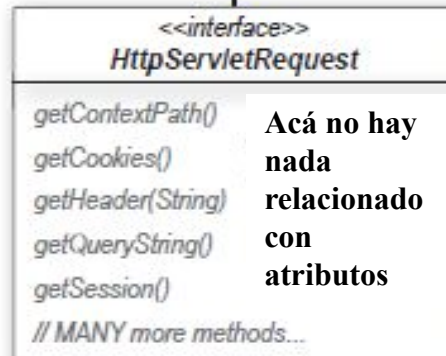
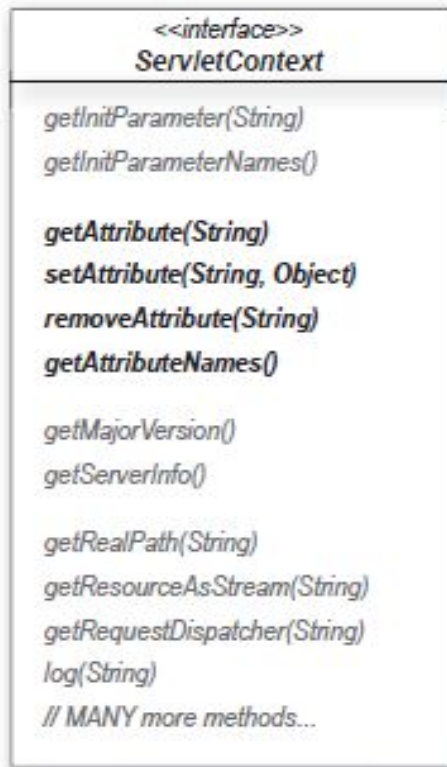
Un atributo es un objeto java guardado dentro de algunos de los siguientes objetos (conocidos como alcances) de la API de servlets: **ServletContext**, **HttpSession** o **HttpServletRequest**.

alcance aplicación	alcance sesión	alcance request
Los objetos ligados a este contexto pueden ser usados por todos los servlets/JSP de la aplicación y duran mientras la aplicación web esté ejecutando.	Los objetos ligados a la sesión de un usuario pueden ser usados por todos los servlets/JSP que accedan a esa sesión y permanecen ahí mientras dure la sesión.	Los objetos ligados al request pueden ser usados por todos los servlets/JSP que dispongan de ese request y permanecen en él, mientras dure el request. Los atributos NO son parámetros.
Se pueden usar los siguientes métodos para ligar, recuperar y eliminar atributos de cualquier alcance. <code>void setAttribute(String nombre, Object attr)</code> <code>Object getAttribute(String nombre)</code> <code>void removeAttribute(String nombre)</code> <code>Enumeration getAttributeNames()</code>		

Notar que el tipo de dato de retorno del método `getAttribute` es `Object` y comúnmente se necesita usar casting para recuperar el tipo original.

Atributos y Alcances

Los atributos pueden ser ligados a uno de los siguientes alcances: **requerimiento**, **sesión** o **aplicación**. Los métodos de la API para ligar atributos son exactamente los mismos.



Acá no hay nada relacionado con atributos



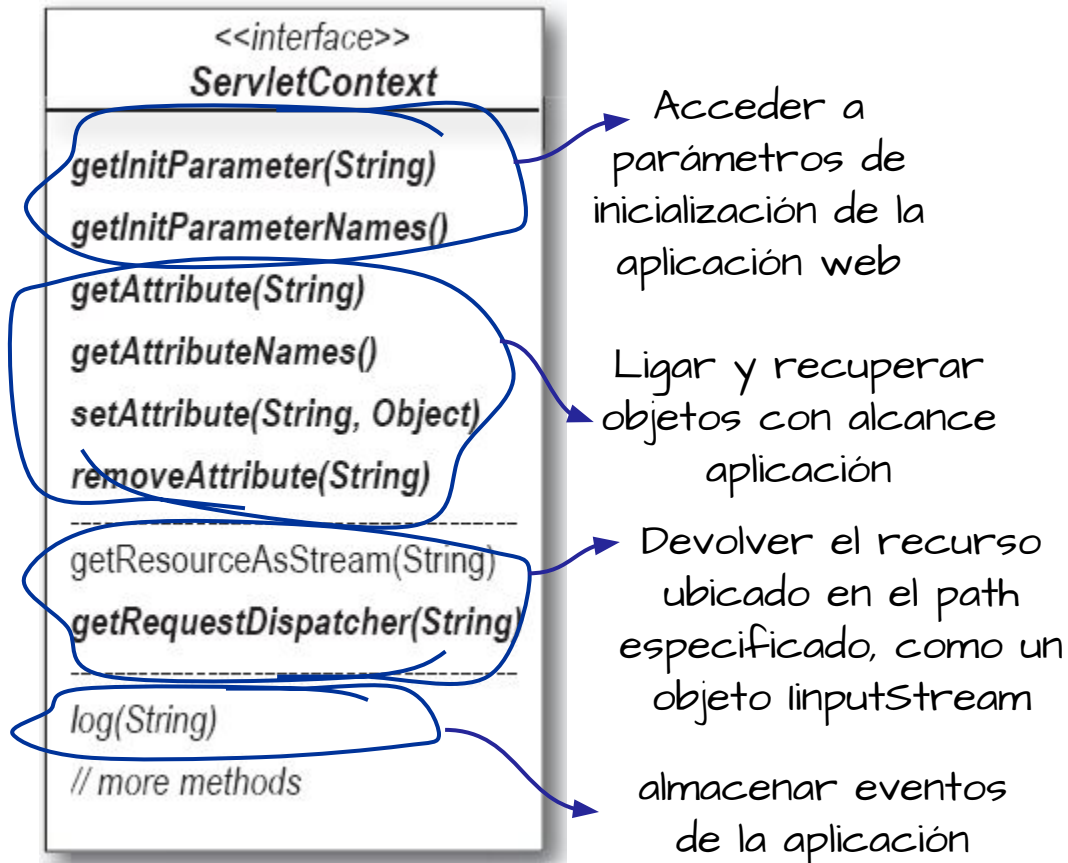
Veremos más adelante

Object **getAttribute(String name)**
void **setAttribute(String name, Object value)**
void **removeAttribute(String name)**
Enumeration **getAttributeNames()**

ServletContext

¿Qué es?

La interfaz **ServletContext** define una vista de la aplicación web para los **servlets**. A través del objeto **ServletContext** se pueden lograr varias funcionalidades observadas en el diagrama de la clase.



Cada Contenedor Web provee una implementación específica de la interfaz **ServletContext**.

Existe un único objeto **ServletContext** por aplicación web ejecutándose en el Contenedor Web.

Los servlets disponen del método **getServletContext()** que retorna una referencia al objeto **ServletContext**. El alcance del objeto **ServletContext** es toda la aplicación.

Para hacer logging de aplicaciones web se recomienda usar una API que permita desde cualquier clase acceder y usar el logging. Ejemplo: **Log4j**, <http://jakarta.apache.org/log4j> o la API estándar para logging del JDK (`java.util.logging`).

ServletContext

Parámetros de Inicialización de la Aplicación Web

De manera similar a los Servlets, las aplicaciones también pueden tener parámetros de inicialización

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns=http://java.sun.com/xml/ns/javaee . . . id="WebApp_ID" version="2.5">
  <context-param>
    <param-name>email</param-name>
    <param-value>admin@info.unlp.edu.ar</param-value>
  </context-param>
  . . .
</web-app>
```

web.xml

Los parámetros de inicialización de la aplicación web pueden ser accedidos desde todas las componentes web.

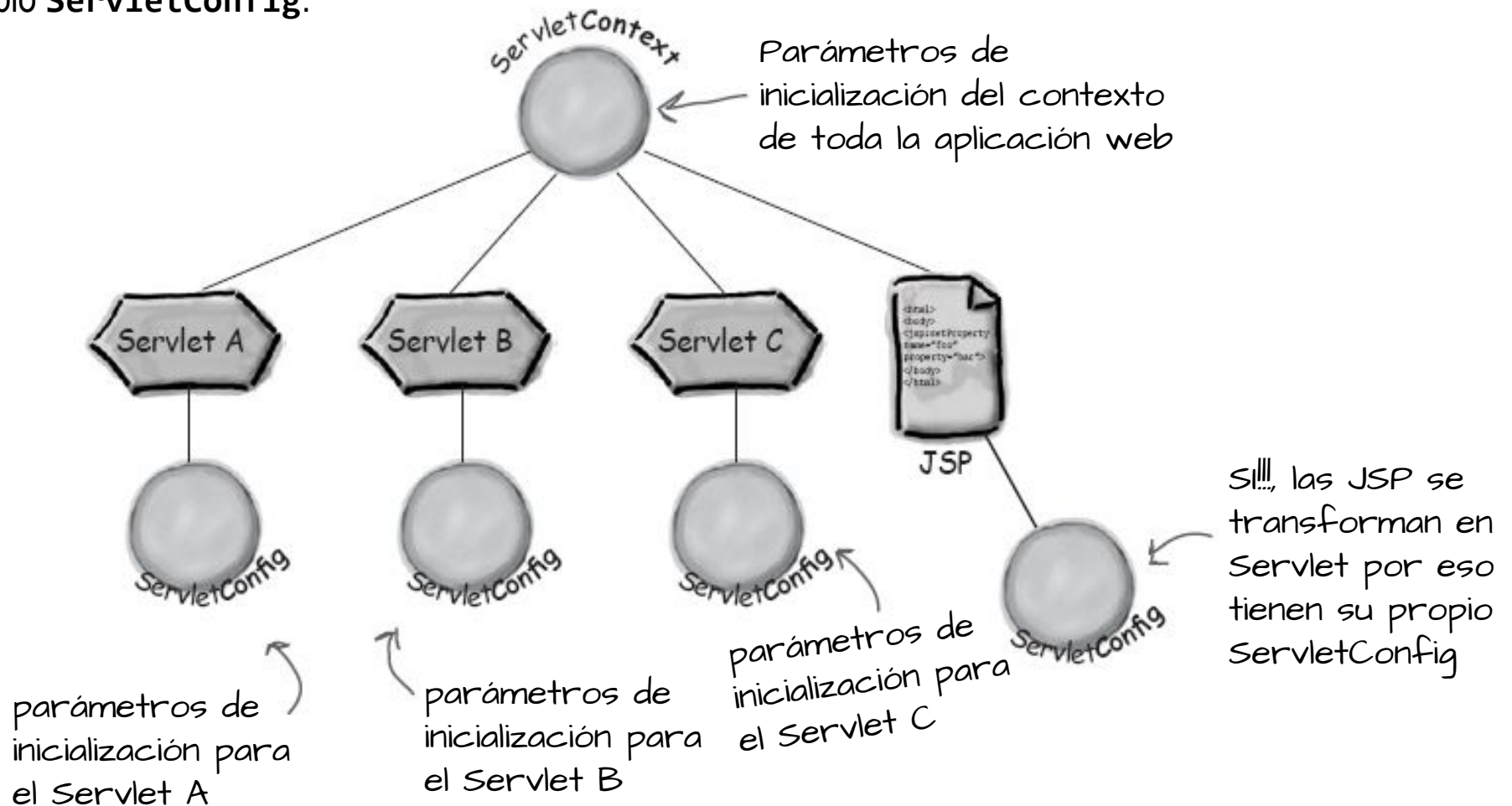
En el ejemplo, el servlet PaginaError está accediendo al parámetro email

```
package com.servlets;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PaginaError extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse resp)..  
        resp.setContentType("text/html");  
        PrintWriter out=resp.getWriter();  
        out.println("<html><head>");  
        out.println("<title> Ocurrió un Error </title>");  
        out.println("</head><body>");  
        ServletContext sc = this.getServletContext();  
        String mail = sc.getInitParameter("email");  
        out.println("<h1> Error inesperado </h1>");  
        out.println("Por favor, reportar a: "+ mail);  
        out.println("</body></html>");  
    }  
}
```

Parámetros de inicialización de Servlets y del Contexto

Hay un único **ServletContext** para toda la aplicación web y todas las componentes lo comparten. El contenedor crea el objeto **ServletContext** cuando se carga la aplicación. Cada servlet en la aplicación tiene su propio **ServletConfig**.



Los parámetros de inicialización

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web
Application 2.3//EN" "http://java.sun.com/j2ee/dts/web-app_2_3.dtd">
<web-app>
```

```
    <context-param>
        <param-name>email</param-name>
        <param-value>admin@info.unlp.edu.ar</param-value>
    </context-param>
```

del Contexto o de la Aplicación

`getServletContext().getInitParameter("email")`

. . .

```
<servlet>
    <servlet-name>ServletFecha</servlet-name>
    <servlet-class>misServlets30.ServletFecha</servlet-class>
    <init-param>
        <param-name>dia</param-name>
        <param-value>Hoy es:</param-value>
    </init-param>
</servlet>
<servlet-mapping>
    <servlet-name>ServletFecha</servlet-name>
    <url-pattern>/ServletFecha</url-pattern>
</servlet-mapping>
. . .
</web-app>
```

del Servlet ServletFecha

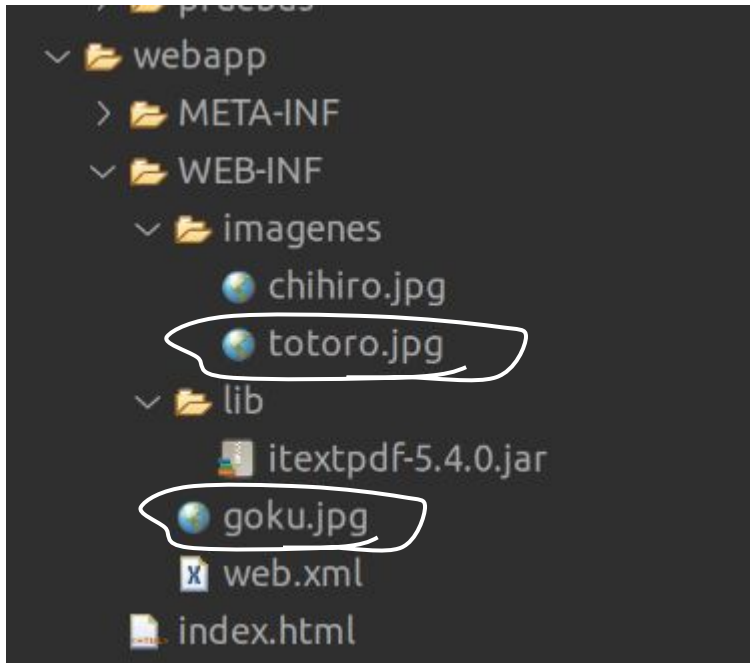
`getServletConfig().getInitParameter("dia")`

web.xml

ServletContext

Recursos web estáticos

El objeto `ServletContext` también provee acceso a la jerarquía de documentos estáticos que forman parte de la aplicación Web, como por ejemplo: archivos TXT, GIF, JPEG.



webapp o context root de Eclipse se traduce al nombre de la aplicación en los contenedores Web. Por ejemplo **cupones**

Nombre de la aplicación web

<https://www.cinemacity.com/cupones>

Paths virtuales de las imágenes:

`/WEB-INF/goku.jpg`

`/WEB-INF/imagenes/totoro.jpg`

`/index.html`

Todos los recursos de una aplicación web son abstraídos en directorios virtuales a partir de la raíz de la aplicación web. Un directorio virtual comienza con `"/` y continúa con un path virtual a directorios y recursos.

ServletContext

Recursos web estáticos

La forma correcta para leer recursos de una aplicación web es usando los métodos **getResource()** o **getResourceAsStream()**. Estos métodos garantizan que el servlet siempre tendrá acceso al recurso deseado. Aún cuando el *deploy* de la aplicación web se realice en múltiples servidores o en un archivo WAR.

El objeto **ServletContext** provee acceso a la jerarquía de documentos estáticos a través de los siguientes métodos:

- **URL getResource(String path)**

Devuelve la URL al recurso que coincide con el **path** dado como parámetro. El **path** debe comenzar con "/" y es relativo a la raíz de la aplicación web.

```
URL unaUrl = getServletContext().getResource("/WEB-INF/goku.jpg") ;
```

- **InputStream getResourceAsStream(String path)**

Devuelve el recurso ubicado en el **path** especificado como parámetro, como un objeto **InputStream**. Los datos en el **InputStream** pueden ser de cualquier tipo y longitud. El path debe empezar con "/" y es relativo a la raíz de la aplicación web.

```
InputStream is=this.getServletContext().getResourceAsStream("/WEB-INF/salas.txt")
```

ServletContext

Recursos web estáticos – Archivos de texto

Utilizando el método `getResourceAsStream()` podríamos leer un archivo de texto ubicado por ejemplo en el directorio **WEB-INF** de la aplicación, de la siguiente manera:

```
@WebServlet("/ServletLeeProperties")
public class ServletLeeProperties extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws . . .{

        String filename = "/WEB-INF/datos.properties";
        ServletContext context = this.getServletContext();

        // Obtenemos un InputStream usando ServletContext.getResourceAsStream()
        InputStream is = context.getResourceAsStream(filename);
        if (is != null) {
            InputStreamReader isr = new InputStreamReader(is);
            BufferedReader reader = new BufferedReader(isr);
            String text;
            // Leemos por linea y escribimos en la salida
            while ((text = reader.readLine()) != null) {
                // Acá se obtiene la linea de texto en la variable text
                // Por ejemplo -> jdbc.driver=com.mysql.jdbc.Driver
            }
        }

        protected void doPost(HttpServletRequest request, HttpServletResponse response) throws . . . {
            doGet(request, response);
        }
    }
}
```

datos.properties

```
jdbc.driver= "com.mysql.jdbc.Driver"
jdbc.user= "admin"
jdbc.password = "tpps@admin2017"
jdbc.url="jdbc:mysql://localhost:3306/myBase"
```

ServletContext

Recursos web estáticos – Archivos de Propiedad

Un archivo de propiedades está formado por líneas de texto de la forma **clave=valor**. La extensión de estos archivos es **properties**.

La clase **ResourceBundle**, del paquete **java.util**, facilita la lectura de archivos de propiedades. Esta clase tiene un método **static getBundle(String name)** que obtiene un objeto **ResourceBundle** usando el nombre del archivo.

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {

    PrintWriter out = response.getWriter();
    ResourceBundle rb = ResourceBundle.getBundle("recursos.datos");
    Enumeration<String> claves = rb.getKeys();
    while (claves.hasMoreElements()) {
        String clave = (String) claves.nextElement();
        out.println(clave+" = "+rb.getString(clave));
    }
    // manualmente -> out.println(rb.getString("jdbc.driver");
}
```

en este caso, el archivo **datos_es_AR.properties** debe ubicarse en la carpeta de fuentes java y carpeta recursos

Estos archivos de texto son comúnmente usados para guardar datos de configuración de una aplicación. A continuación se muestra el contenido de un archivo llamado **datos.properties** y ubicado en **src/recursos**

datos.properties

```
jdbc.driver= "com.mysql.jdbc.Driver"
jdbc.user= "admin"
jdbc.password = "https@admin2017"
jdbc.url="jdbc:mysql://localhost:3306/myBase"
```

ServletContext

Listeners de Contexto

¿Cómo haríamos para correr algún código antes de que un Servlet -o JSP o Filtro- pueda responder a un requerimiento? ¿Podemos saber cuando la aplicación web arranca?

Podemos crear una clase separada (ni Servlet, ni JSP, ni Filtro), que pueda escuchar por 2 eventos del ciclo de vida de la aplicación: **creación** y **destrucción** de la aplicación web. Para lograr esto, la clase debe implementar la interface **ServletContextListener** => Listeners de Contexto.

```
package misServlet;
import javax.servlet.ServletContext;
import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;

public class InicializaSalas implements ServletContextListener {

    public void contextDestroyed(ServletContextEvent sce) { }

    public void contextInitialized(ServletContextEvent sce) {
        // Se leen parametros de inicializacion de la aplicación
        String peli1 = sce.getServletContext().getInitParameter("sala1");
        String peli2 = sce.getServletContext().getInitParameter("sala2");
        String peli3 = sce.getServletContext().getInitParameter("sala3");
        // Se guardan en el contexto, las peliculas en Cartelera
        ServletContext contexto = sce.getServletContext();
        contexto.setAttribute("sala1", peli1);
        contexto.setAttribute("sala2", peli2);
        contexto.setAttribute("sala3", peli3);
    }
}
```



Se ligan los atributos al contexto de la aplicación. Ahora cualquier componente puede recuperarlos usando por ejemplo:

`getServletContext().getAttribute("sala1")`

ServletContext

Listener de contexto – configuración en el WEB.XML

Los **servlet listeners** deben configurarse en el archivo `web.xml` y de esta manera el Contenedor Web se enterará de su existencia. Para publicarlos se utiliza el tag **<listener>**.

`web.xml`

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web
Application 2.3//EN" "http://java.sun.com/j2ee/dts/web-app_2_3.dtd">
<web-app>
  <context-param>
    <param-name>sala1</param-name>
    <param-value>Parasitos</param-value>
  </context-param>
  <context-param>
    <param-name>sala2</param-name>
    <param-value>Joker</param-value>
  </context-param>
  <context-param>
    <param-name>sala3</param-name>
    <param-value>El irlandés</param-value>
  </context-param>
  <listener-class>mislisteners.InicializaSalas</listener-class>
  . . .
</web-app>
```

```
package misListeners;

@WebListener
public class InicializaSalas implements ServletContextListener {

    public void contextDestroyed(ServletContextEvent sce) {
        ..
    }
    public void contextInitialized(ServletContextEvent sce) {
        ...
    }
}
```

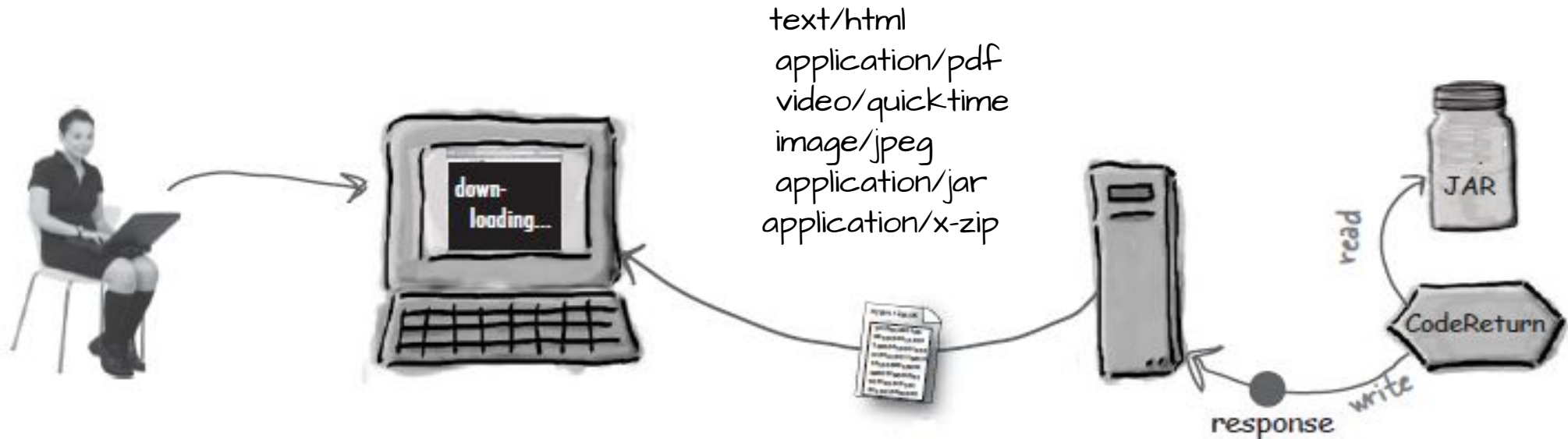
Es posible declarar múltiples listeners.
El orden en que son declarados determina el orden en el que son invocados por el Contenedor Web.

Si usamos anotaciones evitamos el tag **<listener>** en el archivo `web.xml`

Servlets

Tipos de respuestas

Los tipos más comunes de MIME son:



Tenemos un JAR con código fuente java que queremos enviárselo a nuestros clientes usando un Servlet.

¿Qué tipo MIME usamos?

¿Qué objeto usamos para escribir la respuesta?

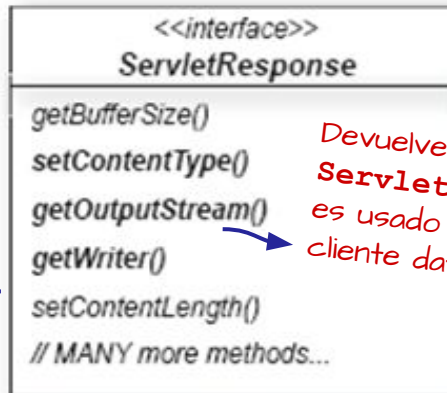
Las interfaces de programación

HttpServletResponse y ServletContext

Vimos que el objeto **HttpServletResponse** tiene métodos que retornan objetos donde se puede escribir el contenido que se enviará en la respuesta.

También existe la interface **ServletContext** que permite obtener referencias a URLs de recursos web.

ServletResponse interface (javax.servlet.ServletResponse)



Devuelve un objeto **PrintWriter** que, es usado por el servlet para escribir la respuesta como texto.

Devuelve un objeto **ServletOutputStream** es usado para enviar al cliente datos binarios.

HttpServletResponse interface (javax.servlet.http.HttpServletResponse)



<<interface>> ServletContext

```
getInitParameter(String)
getInitParameterNames()
getAttribute(String)
getAttributeNames()
setAttribute(String, Object)
removeAttribute(String)
-----
getResourceAsStream(String)
getRequestDispatcher(String)
-----
log(String)
// more methods
```

Cada Contenedor Web provee una implementación específica de esta interface.

La funcionalidad es idéntica, no depende de la implementación, está establecida en la interface.

Devuelve una referencia a un objeto **InputStream** para acceder a un recurso web indicado en el parámetro

Devuelve un objeto **RequestDispatcher** al que se le puede delegar un requerimiento.

Servlets

Tipos de respuestas – Retornando una imagen

Para enviar datos binarios desde un servlet se debe usar el objeto **OutputStream** retornado por el método **getOutputStream()**. En este caso el archivo está en memoria.

```
// imports
@WebServlet("/ServletImage")
public class ServletImage extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        OutputStream outputStream = response.getOutputStream();
        BufferedImage image = new BufferedImage(500, 300, BufferedImage.TYPE_INT_BGR);
        Graphics2D graphics = image.createGraphics();
        graphics.setBackground(Color.WHITE);
        graphics.clearRect(0, 0, 500, 300);
        graphics.setFont(new Font("TimesRoman", Font.BOLD, 14));
        graphics.setColor(Color.DARK_GRAY);
        graphics.drawString("    Comienza tu día con una sonrisa y verás,", 30, 30);
        graphics.drawString("lo divertido que es andar por ahí desentonando", 30, 45);
        graphics.drawString("                                con todo el mundo!!!", 30, 60);

        BufferedImage img = ImageIO.read(this.getServletContext().getResourceAsStream("/WEB-INF/mafalda.png"));
        graphics.drawImage(img, 40, 80, null, null);

        javax.imageio.ImageIO.write(image, "png", outputStream);
        outputStream.close();
    }
}
```



Transferir el control

`sendRedirect()` del objeto `HttpServletResponse`

Algunas veces el servlet puede redireccionar el requerimiento a otro recurso del mismo contenedor web o a una URL de otro dominio.



El `sendRedirect()` hace trabajar al navegador

¿Cómo lo hace?

El servlet invoca al método `sendRedirect(String url)` sobre la respuesta. La respuesta HTTP lleva el **código 302** que indica “El recurso que está buscando el cliente fue temporariamente movido”. El navegador obtiene la respuesta, ve el código de estado “302” genera un nuevo requerimiento usando la URL que recibió como parámetro (el usuario puede observar en la barra del navegador que la URL cambia) y finalmente muestra una página al usuario que no fue la que el originalmente pidió.

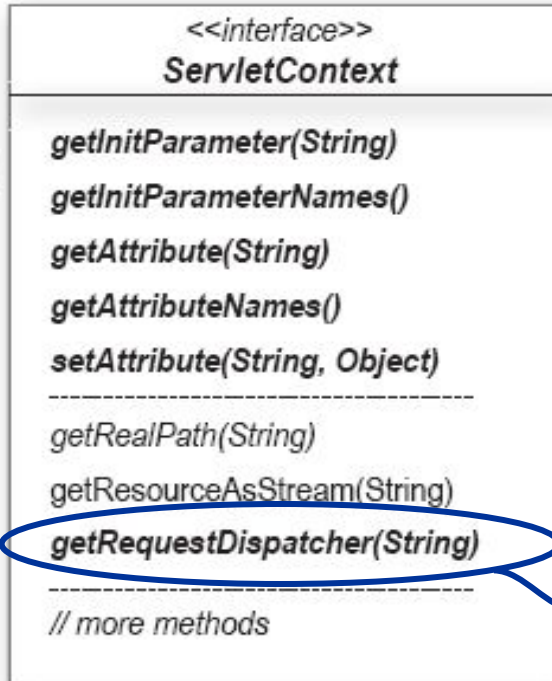
```
public void doPost (HttpServletRequest request, HttpServletResponse response) {  
    String person = request.getParameter("name");  
    if (person==null) {  
        response.sendRedirect("/app/DatosMal.html");  
        return;  
    }  
    . . .  
}
```

La url que se quiere que el navegador use para crear el requerimiento. Es la que verá el cliente

Se puede usar una url relativa o absoluta:
"http://www..."

Transferir el control

forward() del objeto ServletContext



javax.servlet.ServletContext

Usar el método **forward()** del objeto **RequestDispatcher** es otro mecanismo para transferir el control.

A diferencia del redireccionamiento de la respuesta, este mecanismo no requiere de ninguna acción por parte del cliente ni del envío de información extra entre el cliente y el servidor.

El proceso íntegro de delegación del requerimiento se realiza del lado del servidor. Además, este mecanismo permite pasar el requerimiento a otro servlet para que continúe el procesamiento y responda al cliente.



Instancia un
RequestDispatcher
para un servlet

```
public class ForwardServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response) {
        . . .
        RequestDispatcher dispatcher = getServletContext().getRequestDispatcher("/mostrar");
        if (dispatcher != null) {
            request.setAttribute("hora", new Date());
            dispatcher.forward(request, response);
        }
        . . .
    }
}
```

Con el forward() trabaja más el servidor

Transferir el control

`include()` del objeto `ServletContext`

`<<interface>>`
`RequestDispatcher`

`forward(ServletRequest, ServletResponse)`
`include(ServletRequest, ServletResponse)`

El objeto `RequestDispatcher` también cuenta con el método `include()` que se utiliza de manera similar que el `forward()` y que incluye contenido del lado del servidor en la respuesta que se está generando. El servlet que funciona como receptor del método `include()` – en el ejemplo los servlets `header` y `footer`– tienen acceso al requerimiento y a la respuesta original.

```
public class IncludeServlet extends HttpServlet {  
    public void doGet(HttpServletRequest req, HttpServletResponse res) {  
        . . .  
        RequestDispatcher dispatcher = getServletContext().getRequestDispatcher("/header");  
        if (dispatcher != null)  
            dispatcher.include(request, response);  
        . . .  
        dispatcher = getServletContext().getRequestDispatcher("/footer");  
        if (dispatcher != null)  
            dispatcher.include(request, response);  
    }  
    . . .  
}
```

Acá se podrían setear atributos en el request antes de delegar

El archivo descriptor de la Aplicación Web

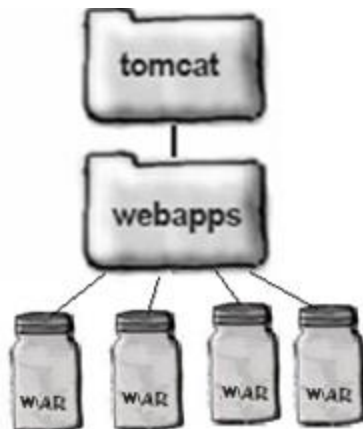
- El archivo descriptor de la aplicación web, **web.xml**, define TODO lo que el servidor necesita conocer sobre la aplicación web.
- Es estándar y se ubica SIEMPRE en la carpeta **/WEB-INF/web.xml**.
- La especificación de Servlets incluye un Document Type Descriptor (DTD) para el **web.xml** que define su gramática. Por ej. los elementos descriptores `<filter>`, `<servlet>` y `<servlet-mapping>` deben ser ingresados en el orden establecido por el DTD. En general los contenedores fuerzan estas reglas cuando procesan los archivos **web.xml**
- Al ser declarativa la información contenida en el archivo **web.xml** es posible modificarla sin necesidad de modificar el código fuente de las componentes.
- En ejecución, el contenedor web lee el archivo **web.xml** y actúa en consecuencia.

Los IDEs (Eclipse, IntelliJ, JDeveloper, etc.) proveen editores visuales y ayudas durante el desarrollo de la aplicación web, que permiten crear, actualizar y editar en forma simple y consistente el web.xml.

¿Cómo se hace el “deploy” de una aplicación?

- Las aplicaciones web JAVA pueden empaquetarse en un archivo *Web ARchive* (WAR). El archivo WAR es ideal para distribuir e instalar una aplicación. El formato “desempaquetado” es útil en la **etapa de desarrollo**.
- Un WAR tiene una estructura de directorios específica, donde la raíz, es el “*context root*” de la aplicación web.
- El archivo WAR es un archivo JAR que contiene un módulo web: páginas HTML, archivos de imágenes, JSPs, clases, páginas de estilo, código JavaScript, el directorio **WEB-INF** y sus subdirectorios (classes, lib, tag, el archivo web.xml, etc.)
- Los archivos WAR están definidos oficialmente en la especificación de Servlets a partir de la versión 2.2. **Son estándares**. Todos los contenedores que implementan la especificación de la API de Servlets 2.2 y superiores deben soportar archivos WAR.
- Los IDEs proveen opciones que permiten construir el WAR en forma automática. Se puede crear el archivo WAR usando la herramienta *jar* del JDK.

En el servidor Tomcat, el archivo WAR de la aplicación web se debe copiar en el directorio **webapps**



Cuando Tomcat arranca, automáticamente expande a partir de webapps el contenido de cada uno de los archivos .war al formato “desempaquetado”.

Si usamos esta técnica para hacer el “deployment” de nuestra aplicación y necesitamos actualizarla, debemos reemplazar el .WAR y ELIMINAR la estructura de directorios expandida y luego re-iniciar Tomcat.

El Módulo Web

Un módulo web es una unidad “*desplegable*” de recursos web (componentes web y archivos estáticos que pueden referenciarse por una URL). También puede contener clases utilitarias “*server-side*” (por ej: javaBeans) y clases “*client-side*” (applets y clases utilitarias).

De acuerdo a la especificación de Servlets, un módulo web se corresponde con una aplicación web y tiene la siguiente estructura de directorios:

